Taller: Desarrollo de APIs REST en Spring Boot con Swagger

Objetivo:

Aprender a construir una API REST con Spring Boot que maneje datos en memoria utilizando listas de objetos, sin bases de datos, e integrando Swagger para la documentación y pruebas de la API.

Duración: 3 horas

Requisitos Previos:

JDK 17 o superior instalado
IDE como IntelliJ IDEA, Eclipse o VS Code con Extensión de Java
Navegador Web (Chrome, Edge, Firefox) para probar Swagger
Conocimientos básicos de Java y programación orientada a objetos

Parte 1: Configuración del Proyecto en Spring Boot

Ejercicio 1: Creación del Proyecto Spring Boot

Objetivo: Configurar un proyecto base con Spring Boot y Swagger

Paso a Paso:

1. **Ve a Spring Initializr y genera un nuevo proyecto con las siguientes opciones:**
    - **Project: Maven**
    - **Language: Java**
    - **Spring Boot Version: 3.x.x**
    - **Dependencies:**
      Spring Web
      Spring Boot DevTools
      Springdoc OpenAPI (para Swagger)
2. **Descarga y descomprime el proyecto.**
3. **Abre el proyecto en tu IDE (IntelliJ o Eclipse).**
4. **Ejecuta la aplicación con mvn spring-boot:run o ejecutando la clase MainApplication.java.**
5. **Abre el navegador y accede a http://localhost:8080 para verificar que la aplicación corre correctamente.**

Resultado esperado: Proyecto Spring Boot con Swagger listo para desarrollo.

Parte 2: Creación de una API REST con Datos en Memoria y Swagger

Ejercicio 2: Agregar Swagger para Documentación de la API

Objetivo: Configurar Swagger para documentar y probar la API

Paso a Paso:

1. **Abre el archivo pom.xml y agrega la dependencia de Springdoc OpenAPI:**

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>
```

2. **Guarda los cambios y ejecuta la aplicación nuevamente con mvn spring-boot:run.**
3. **Abre el navegador y accede a http://localhost:8080/swagger-ui.html para ver la interfaz de Swagger.**

Resultado esperado: Swagger funcionando y listo para probar la API.

Ejercicio 3: Creación del Modelo y la Lista de Datos

Objetivo: Crear una clase Producto y una lista en memoria para simular la base de datos.

Paso a Paso:

1. Dentro de src/main/java/com/miempresa/api, crea un paquete llamado models.
2. Dentro de models, crea la clase Producto.java:

```
package com.miempresa.api.models;

import io.swagger.v3.oas.annotations.media.Schema;


@Schema(description = "Entidad Producto que representa un artículo en la tienda")
```

```
public class Producto {
  @Schema(description = "ID único del producto", example = "1")
  private Long id;

  @Schema(description = "Nombre del producto", example = "Laptop")
  private String nombre;

  @Schema(description = "Precio del producto", example = "1500.99")
  private double precio;

  public Producto(Long id, String nombre, double precio) {
    this.id = id;
    this.nombre = nombre;
    this.precio = precio;
  }
  public Long getId() { return id; }
  public void setId(Long id) { this.id = id; }

  public String getNombre() { return nombre; }
  public void setNombre(String nombre) { this.nombre = nombre; }

  public double getPrecio() { return precio; }
  public void setPrecio(double precio) { this.precio = precio; }
}
```

Resultado esperado: Clase Producto creada con anotaciones de Swagger para documentación.

Ejercicio 4: Creación del Controlador REST

Objetivo: Implementar un CRUD con una lista en memoria y documentar la API con Swagger.

Paso a Paso:

1. Dentro de src/main/java/com/miempresa/api, crea un paquete llamado controllers.
2. Dentro de controllers, crea la clase ProductoController.java:

```
package com.miempresa.api.controllers;


import com.miempresa.api.models.Producto;

import io.swagger.v3.oas.annotations.Operation;

import io.swagger.v3.oas.annotations.tags.Tag;

import org.springframework.web.bind.annotation.*;


import java.util.ArrayList;

import java.util.List;


@RestController

@RequestMapping("/productos")

@Tag(name = "Productos", description = "API para gestionar productos en memoria")

public class ProductoController {

  private List<Producto> productos = new ArrayList<>();


  public ProductoController() {

    productos.add(new Producto(1L, "Laptop", 1500.99));

    productos.add(new Producto(2L, "Mouse", 25.50));

    productos.add(new Producto(3L, "Teclado", 45.75));

  }
```

```
@GetMapping
 @Operation(summary = "Obtener todos los productos", description = "Devuelve una lista de productos almacenados en memoria")
  public List<Producto> listarProductos() {
    return productos;
  }

  @PostMapping
  @Operation(summary = "Crear un nuevo producto", description = "Agrega un producto a la lista en memoria")
  public Producto crearProducto(@RequestBody Producto producto) {
    producto.setId((long) (productos.size() + 1)); // Asignar un ID único
    productos.add(producto);
    return producto;
  }

  @PutMapping("/{id}")
  @Operation(summary = "Actualizar un producto", description = "Modifica un producto existente en memoria")
  public Producto actualizarProducto(@PathVariable Long id, @RequestBody Producto producto) {
    for (Producto p : productos) {
      if (p.getId().equals(id)) {
        p.setNombre(producto.getNombre());
        p.setPrecio(producto.getPrecio());
        return p;
      }
    }
    return null;
  }
```

```
@DeleteMapping("/{id}")
@Operation(summary = "Eliminar un producto", description = "Elimina un producto de la lista en memoria")
  public String eliminarProducto(@PathVariable Long id) {
    productos.removeIf(p -> p.getId().equals(id));
    return "Producto eliminado";
  }
}
```

Resultado esperado: API con documentación Swagger y un CRUD funcional en memoria.

Parte 3: Pruebas con Swagger

Prueba los siguientes endpoints accediendo a http://localhost:8080/swagger-ui.html:

Método	URL	Descripción
GET	/productos	Obtener lista de productos.
POST	/productos	Crear un nuevo producto.
PUT	/productos/{id}	Actualizar un producto existente.
DELETE	/productos/{id}	Eliminar un producto.

Ejemplo de cuerpo JSON para POST /productos:

```
{
  "nombre": "Monitor",
  "precio": 299.99
}
```

Resultado esperado: Swagger muestra y permite probar la API directamente desde su interfaz gráfica.

Parte 4: Reto Final

Reto:

1. Agregar validaciones en ProductoController para evitar valores nulos o negativos.
2. Añadir un endpoint de búsqueda (GET /productos?nombre=Laptop).
3. Personalizar Swagger cambiando el título y la descripción en application.properties.

Conclusión

Construimos una API REST en Spring Boot sin base de datos.
Implementamos un CRUD en memoria.
Probamos la API en Swagger.

Siguientes pasos: Integrar esta API con Spring Security para autenticación.